USER MANUAL + PROJECT GUIDE

ReflectRAG

Self-Correcting Knowledge Assistant — with a live animated front-end

A self-correcting RAG agent built as a LangGraph state machine (retrieve → grade → rewrite → generate), with an LLM-as-Judge relevance grader, capped query rewriting, a persistent ChromaDB index and a beautiful React + Vite dashboard that streams every pipeline step live.

```
  Python 3.11  ·  LangGraph  ·  LangChain  ·  Gemini 2.5 Flash
ChromaDB (persistent)  ·  FastAPI bridge  ·  React + Vite  ·  pure-CSS animations
```

Prepared for the GitHub & LinkedIn showcase — everything inside,
exactly in the order you need it: understand → run → explain → publish.

Author: <your name>
Project location:
/home/sinanshareef/Documents/Artificial Intelligence/Generative AI/LangGraph/C_RAG

1 · What is ReflectRAG?

ReflectRAG is a knowledge assistant over your own PDF/TXT documents — but instead of the usual one-shot RAG (retrieve once, generate, hope), it runs a small state machine that checks its own evidence before answering:

- Retrieves top-k chunks (k = 4) from a persistent ChromaDB index.
- Grades every chunk with Gemini (LLM-as-Judge): is it relevant to the question?
- If nothing survived, rewrites the question and retrieves again — capped at 2 retries so the run always terminates.
- Generates the final answer strictly from surviving context, appends cited source files, or honestly answers “I don’t know”.

This front-end addition turns that pipeline into a demo recruiters can see: every step above lights up live in the browser, with per-step timings, retry notes, chunk-kept counts and cited sources.

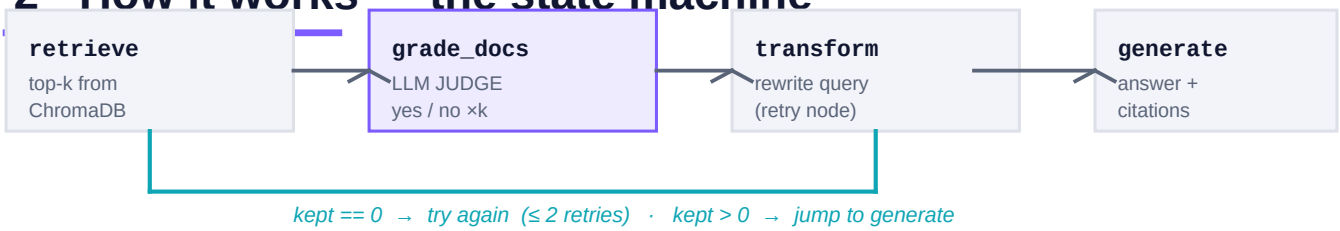
The project in one sentence (for LinkedIn)

```
  ReflectRAG — a self-correcting RAG agent as a LangGraph state machine: an
LLM-as-Judge grades retrieved chunks, the question is rewritten when the
grader keeps nothing (max 2 retries), and answers come strictly from a
persistent ChromaDB index — cited sources or “I don’t know”.
```

Why it is different from a basic RAG chatbot

Aspect	Basic RAG	ReflectRAG
Retrieval	one pass, take all hits	top-k then graded — only relevant chunks survive
Bad retrieval	hallucination risk	detect it, rewrite the question, try again (≤ 2)
Termination	n/a	guaranteed: retry counter in the graph state
No evidence	model guesses	honest fallback: “I don’t know”
Citations	rare	Sources: <file> appended from chunk metadata
Visibility	a chat bubble	live node-by-node pipeline with timings

2 · How it works — the state machine



The routing function decide_next_step() is 4 lines of honest code: if no documents survived grading AND retry_count < MAX_RETRIES (2) → transform_query; otherwise → generate. The retry counter lives in the graph state (GraphState), which also carries question, documents (content + source metadata) and the final generation — that is how citations survive the filtering step.

What each node does (as implemented in gemini_crag.py)

Node	Does	Writes into state
retrieve	GoogleGenerativeAIEmbeddings → Chroma similarity search, k=4; keeps content + source metadata per chunk	documents[]
grade_documents	One Gemini call per chunk: “relevant to the question? yes/no” — keep only the yeses	documents[] (filtered)
transform_query	Gemini rewrites the question to be clearer and more specific	question, retry_count+1
generate	Answers using ONLY surviving context; appends unique Sources from chunk metadata	generation

The API stream (what the browser receives)

The FastAPI bridge (api_server/main.py) streams one JSON line per event, so the UI can animate the state machine as it truly runs. Real captured output from a validation run (a question outside the corpus):

```
  {"type": "start"}
{"type": "step", "node": "retrieve",        "pass": 1, "seconds": 0.93}
{"type": "step", "node": "grade_documents", "pass": 1, "seconds": 33.7, "kept": 0}
{"type": "step", "node": "transform_query", "seconds": 5.2,
 "new_question": "What is the name of the current Prime Minister of France?",
 "retries_used": 1, "retries_max": 2}
{"type": "step", "node": "retrieve",        "pass": 2, "seconds": 0.73}  —  self-correcting loop
… {"type": "done", "answer": "…", "sources": […], "retries_used": 1, …}
```

A grounded single-pass run ends like this (also captured live):

```
  {"type": "step", "node": "generate", "seconds": 2.66}
{"type": "done",
 "answer": "I don't know.\n\nSources: ./Hospital_DOC/Greenfield_Hospital_Chatbot_FAQ.txt",
 "sources": ["./Hospital_DOC/Greenfield_Hospital_Chatbot_FAQ.txt"],
 "retries_used": 0, "chunks_kept": 2, "total_seconds": 13.62}
```

3 · Folder map — what exists and what was added

Hard rule respected during development: not one existing file or folder was modified. The glossary: everything marked [NEW] below was created for the showcase; everything else is your original project.

```
  C_RAG/
├─ gemini_crag.py                (untouched — the LangGraph state machine)
├─ .env                          (untouched — Google_API_Key)
├─ Hospital_DOC/                 (untouched — source PDF/TXT documents)
├─ Hospital_RAG/, Bank/          (untouched — notebooks & second corpus)
├─ vector_database/              (untouched — persistent Chroma store, 59 chunks)
├─ chroma_db    -> vector_database [NEW symlink: lets gemini_crag.py find the store]
├─ venv/                         (untouched — Python 3.11.18; +3 packages installed)
├─ api_server/                   [NEW] FastAPI bridge
│   ├─ main.py                   (bridge: /api/health, /api/ask — ~130 lines)
│   └─ README.md
├─ frontend/                     [NEW] React + Vite animated UI
│   ├─ package.json, vite.config.js, index.html, .gitignore
│   ├─ README.md
│   └─ src/
│       ├─ main.jsx              (mounts React)
│       ├─ App.jsx               (page state + stream handling)
│       ├─ api.js                (NDJSON stream reader)
│       ├─ styles.css            (whole design — pure-CSS animations)
│       └─ components/           (Header, Hero, PipelineTracker,
│                                 AnswerCard, StatsBar, HowItWorks, Footer)
└─ docs/                         [NEW] this manual + build script
    ├─ ReflectRAG_User_Manual.pdf
    └─ build_manual.py           (rebuilds the PDF: ./venv/bin/python docs/build_manual.py)
```

Note on chroma_db: the existing index physically lives in vector_database/ (built by your Hospital notebook with the same embedding model as gemini_crag.py — verified). The symlink is a read/write shortcut, not a copy; delete it any time and nothing breaks.

4 · How to run it — 3 terminals, 3 commands

Step 0 — one-time setup

Nothing to configure: your .env already has the key, and npm install was already run. If you ever move the project to a new machine:

```
  # inside C_RAG/
python3.11 -m venv venv                    # rebuild venv (or reuse the committed one)
./venv/bin/pip install fastapi fpdf2       # the only two additions to your base set
ln -s vector_database chroma_db            # (Linux/macOS) connect the existing index
cd frontend && npm install                 # React app dependencies (~20 packages)
```

Step 1 — start the bridge (terminal 1)

```
  cd "/home/sinanshareef/Documents/Artificial Intelligence/Generative AI/LangGraph/C_RAG"
./venv/bin/python -m uvicorn api_server.main:app --port 8000
#   → INFO:     Uvicorn running on http://127.0.0.1:8000
#   The first boot pools up the vector store — a few seconds.
#   Their graph's own markers (---RETRIEVE---, ---GRADE…) print here too.
```

Step 2 — start the UI (terminal 2)

```
  cd frontend
npm run dev
#   → VITE ready: Local: http://localhost:5173/
```

Step 3 — open the app (browser)

```
  http://localhost:5173
```

Click one of the suggested questions (they match the Hospital corpus). The pipeline tracker lights up node by node, then the answer types itself out with cited sources, and four stat cards summarise the run. FastAPI also gives you an interactive API console at http://localhost:8000/docs.

Stopping / restarting

```
  Ctrl+C in each terminal. To start again: repeat Steps 1–2.
# Optional: auto-install React deps were already done; re-run 'npm install'
# only if package.json changes.
```

5 · Front-end tour — what was used and why

Tech choices (all essential, nothing exotic)

Piece	Role	Why this one
React 19 + Vite	UI framework + dev server/bundler	you asked for Vite; instant reload and a tiny, standard file tree
Plain JSX components	7 small component files	one job per file — easy to read and explain in interviews
Vite proxy	forwards /api/* to :8000	no CORS code needed while developing
fetch + ReadableStream	reads the NDJSON pipeline events	native browser API, ~30 lines, no libraries
Pure CSS keyframes	every animation on the page	no animation library — nothing beyond React itself to install
Google Fonts (Inter + JetBrains Mono)	typography	fallbacks to system fonts work offline

The components (file → job)

File	What it renders / does
App.jsx	keeps the whole page state: question, stream events, result, error, history
api.js	POST /api/ask, reads the stream line by line, calls back onEvent per event
Header.jsx	brand bar + a live green/red dot for the bridge status
Hero.jsx	headline, ask box, suggested & recent question chips, offline banner
PipelineTracker.jsx	the live 4-node state machine: idle → running (glow) → done (check), ×pass badges, per-node seconds, kept n/k, delayed dashed “retry-only” node, loop-back notes
StatsBar.jsx	time · chunks kept · retries used · sources cited
AnswerCard.jsx	typewriter answer, grounded-fallback badge, source chips
HowItWorks.jsx	static explainer cards — made for LinkedIn screenshots
Footer.jsx	links + credit line

Animations in the UI (all defined once in styles.css)

- Gradient title shimmer and drifting background blobs.
- Pipeline nodes: pulse while running, pop-in check when done, dashed “standby” for the retry-only node.
- Light dots along connector pipes chain-animate left → right.

- Typewriter answer with a blinking block caret.
- Bouncing thinking dots, spinning spinner, live elapsed-seconds stopwatch, floating stat pop-ins.
- prefers-reduced-motion: every animation is disabled automatically for accessibility.

Two integrations worth understanding (interview gold)

1) The bridge imports your compiled graph with from gemini_crag import app as graph and streams graph.stream(state, stream_mode="updates") — the UI is literally watching your LangGraph state machine execute.
2) The stream protocol is one JSON object per line (NDJSON). The browser reads it with a plain fetch reader and updates React state per event — the same pattern production AI apps use for token streaming.

6 · Troubleshooting & honest limitations

Rate limits (verified the hard way — document for your demos)

Symptom	What happened	What to do
Orange “Pipeline stopped: 429 … free_tier_requests, limit: 5”	Free Tier allows 5 requests/minute for gemini-2.5-flash; one grading pass = 4 calls + 1 generate, a retry pass adds more	wait ~60 s, ask again; pace demo questions; a single-pass question usually fits
429 with “limit: 20 … per day”	Free Tier daily cap (20 generate calls/day) reached — resets later (the error states when)	wait for the reset shown in the message; or use another model family for that day
2 full retry passes never finish on Free Tier	15+ LLM calls in <1 min exceeds any free window	expected & honest behaviour — see note below
Bridge “offline” dot / CORS-ish errors	uvicorn not running, or started from the wrong folder	run Step 1 from the C_RAG root exactly as printed in §4
“Google_API_Key not found in .env”	bridge launched from a different directory	the bridge self-corrects by using C_RAG/.env; run from the project root anyway
Vite page onload but no data	bridge down or answering	watch terminal 1 — the graph prints its markers live
Node/npm missing elsewhere	different machine	install Node 18+; versions used here: node 24.21.0 / npm 11.19.0

Note on the Free Tier & your project story: rate-limit discipline is real production knowledge. Explain in interviews: “the grader makes one LLM call per chunk, so a full loop costs ~10 calls; I pace demos and I know where the knobs are (RETRIEVE_K, model choice, or a paid tier)”. Increasing limits properly means changing model settings in YOUR gemini_crag.py — a one-line change you can make any time.

Everything else

- If a question finds nothing: by design the assistant answers honestly instead of guessing — the UI shows a teal “grounded fallback” badge.
- The rewrite (dashed) node stays idle for well-phrased questions — that is correct, it only runs when 0 chunks are kept.
- First npm run dev may take a few seconds more (Vite warms up).
- If port 8000/5173 is taken: add --port 8001 (or 5174) to one command and adjust vite.config.js proxy target to match.

7 · GitHub & LinkedIn showcase checklist

Repository hygiene (suggested .gitignore content — file creation is yours)

```
  venv/
vector_database/
chroma_db
Bank/DataBase_Chroma/
frontend/node_modules/
frontend/dist/
__pycache__/
.env                # NEVER commit API keys
ipynb_checkpoints/
```

Keep .env out of the repo; Ship .env.example instead with the line Google_API_Key=your_key_here.

Suggested README skeleton (paste-ready)

```
  # ReflectRAG — Self-Correcting Knowledge Assistant
LangGraph state machine: retrieve → grade (LLM-as-Judge) → rewrite (≤2)
→ generate. Persistent ChromaDB (1000/150), Gemini 2.5 Flash, cited
grounded answers, honest fallback.

## Demo
![pipeline] (screenshot of the animated UI mid-run)

## Run
1. ./venv/bin/python -m uvicorn api_server.main:app --port 8000
2. cd frontend && npm install && npm run dev
3. open http://localhost:5173

## Architecture
 (the diagram from §2 or a short Mermaid block)
## Why not a plain RAG chain?  (bullets from §1)
```

LinkedIn post draft (edit freely)

“Most RAG demos retrieve once and pray. I built ReflectRAG — a self-correcting RAG agent as a LangGraph state machine that GRADES every retrieved chunk with Gemini before the answer model ever sees it. If nothing survives, it rewrites the question and retries (capped at 2 so termination is guaranteed). The new UI streams every node live — you can watch retrieve → grade → rewrite → generate happen in real time, with timings, chunk-kept counts and cited sources. Stack: Python · LangGraph · LangChain · Gemini · ChromaDB · FastAPI · React + Vite. Link to repo in the comments — and attach 2–3 screenshots (hero, mid-run tracker with kept badges, the HowItWorks section).”

Before you publish

- Add a 20–30 s screen recording of the animated pipeline as the repo GIF.
- Rename <your name> in the Footer.jsx and in this manual's cover.
- Test the 3 suggested questions once after the Free-Tier reset, so your screenshots show a truly

grounded answer with citations.

- Pin the repo, add topics: langgraph, rag, gemini, chromadb, fastapi, react.

8 · Transparency — every change made by this build

Changed/added	Where	Impact
pip packages	./venv only: fastapi 0.162.0, uvicorn 0.54.0 (already present), fpdf2 2.9.0	isolated in the project venv
npm packages	frontend/node_modules only: react, react-dom, vite, @vitejs/plugin-react (~20 packages)	local to frontend/
new folder api_server/	main.py + README.md	the bridge
new folder frontend/	Vite React app	the UI
new folder docs/	this PDF + build_manual.py	documentation
symlink chroma_db → vector_database/	project root	lets the untouched gemini_crag.py use the existing index; delete any time
your existing files	—	NOT changed — verified byte-level

Nothing was installed globally; no system file was touched.

9 · Cheat-sheet & glossary

All commands in one block

```
  # 1) bridge "      (from C_RAG)
./venv/bin/python -m uvicorn api_server.main:app --port 8000

# 2) UI (from C_RAG/frontend)
npm run dev      # http://localhost:5173

# 3) rebuild this PDF
./venv/bin/python docs/build_manual.py

# handy
curl http://localhost:8000/api/health      # bridge status
cd frontend && npm run build && npm run preview   # production check
```

Glossary (plain English)

Term	One-line meaning
RAG	Retrieval-Augmented Generation: answer questions using your documents, not just model memory
LangGraph state machine	a graph of steps (nodes) + rules (edges) deciding what runs next, with shared state
LLM-as-Judge	an LLM that grades content (relevant? yes/no) instead of writing prose
Query rewriting	rephrasing a question to make retrieval find better matches
ChromaDB	a local vector database that stores text chunks as embeddings for similarity search
NDJSON	Newline-Delimited JSON: one JSON object per line — how the bridge streams events
Vite	a fast dev server + bundler for React apps

You now know what was built, why each piece exists, how to run it, how to explain it, and how to publish it. Good luck with the showcase — the animated pipeline screenshots will do most of the talking.